

Le Perceptron Multicouche (MLP)

Des Réseaux de Neurones aux Frontières Non-Linéaires

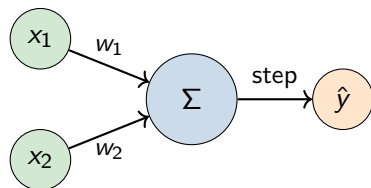
- 1 Rappel et Motivation
- 2 Architecture du MLP
- 3 Fonctions d'Activation Non-Linéaires
- 4 Le Modèle : Propagation Avant
- 5 L'Algorithme : Rétropropagation

Rappel : Le Perceptron Simple

Le Perceptron est un classifieur **linéaire** :

- Somme pondérée : $z = w^T x + b$
- Activation en escalier (step function)
- Frontière = une droite (ou hyperplan)

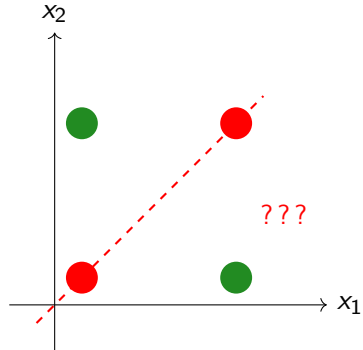
Limitation : Ne peut résoudre que des problèmes **linéairement séparables**.



Le Problème XOR : Rappel

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

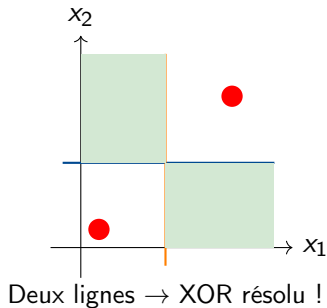
Impossible de séparer avec une seule droite !



La Solution : Combiner Plusieurs Neurones

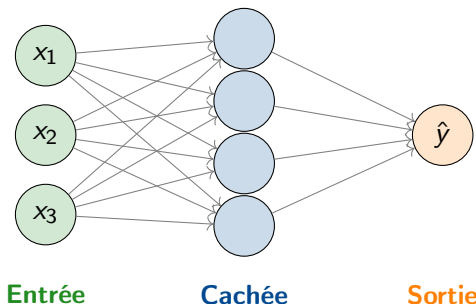
L'Idee Clé

Si un neurone trace une ligne, **plusieurs neurones** peuvent tracer plusieurs lignes et les combiner !



Architecture du MLP

Structure en Couches



Les Trois Types de Couches

- 1 **Couche d'Entrée** : Reçoit les données brutes (pas de calcul)
- 2 **Couches Cachées** : Transforment les entrées (la "magie")
- 3 **Couche de Sortie** : Produit la décision finale

Connexions Denses (Fully Connected)

Définition

Un MLP est généralement **entièrement connecté** :

- Chaque neurone de la couche l est connecté à **tous** les neurones de la couche $l + 1$
- Chaque connexion a son propre poids

Nombre de paramètres entre deux couches de tailles n_l et n_{l+1} :

$$\text{Poids} + \text{Biais} = n_l \times n_{l+1} + n_{l+1}$$

Exemple

Entrée (784) $\rightarrow$ Cachée (128) $\rightarrow$ Sortie (10)

Paramètres : $(784 \times 128 + 128) + (128 \times 10 + 10) = 101,770$

Profondeur vs Largeur

Réseau Large (shallow)

- Peu de couches cachées
- Beaucoup de neurones par couche
- Apprend des motifs "simples"

Réseau Profond (deep)

- Beaucoup de couches cachées
- Moins de neurones par couche
- Apprend des **hiérarchies** de caractéristiques

Deep Learning

Les réseaux **profonds** (2+ couches cachées) sont à la base du Deep Learning moderne.

Fonctions d'Activation L'Ingrédient Secret

Pourquoi la Non-Linéarité ?

Le Problème

Si on empile des transformations **linéaires**, le résultat reste linéaire !

$$f(g(x)) = W_2(W_1x + b_1) + b_2 = W'x + b'$$

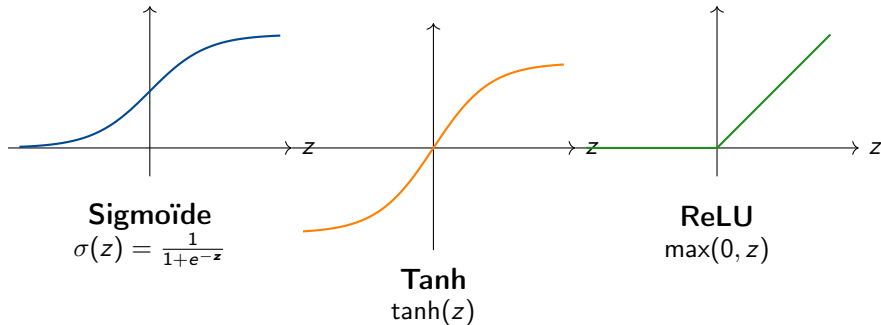
La Solution

Ajouter des **fonctions d'activation non-linéaires** après chaque couche !

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

C'est ce qui permet au réseau d'apprendre des frontières **courbes et complexes**.

Les Fonctions d'Activation Classiques



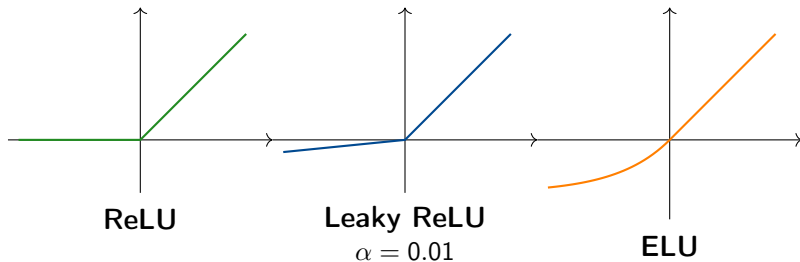
Comparaison des Fonctions d'Activation

	Sigmoïde	Tanh	ReLU
Sortie	$[0, 1]$	$[-1, 1]$	$[0, +\infty[$
Centrée	Non	Oui	Non
Gradient	Peut s'évanouir	Peut s'évanouir	Stable si $z > 0$
Calcul	Coûteux (exp)	Coûteux (exp)	Très rapide

ReLU : Le Standard Moderne

- Calcul très rapide : juste $\max(0, z)$
- Pas de problème de "gradient qui s'évanouit" (pour $z > 0$)
- Utilisé dans la majorité des réseaux modernes

Variantes de ReLU



- **Leaky ReLU** : Pente faible pour $z < 0$ (évite les "neurones morts")
- **ELU** : Exponentielle pour $z < 0$ (sorties centrées)

Rappel : Les 3 Composants d'un Algorithme ML

Pour entraîner un MLP, nous devons définir :

- ❶ **Le Modèle** : Comment calculer la prédiction ?
 - → **Propagation avant**
- ❷ **Fonction de Coût** : Comment mesurer l'erreur ?
 - → MSE, Cross-Entropy (déjà vu)
- ❸ **L'Algorithme** : Comment mettre à jour les poids ?
 - → **Rétropropagation**

Plan

Nous allons maintenant détailler le **Modèle** et l'**Algorithme**.

Le Modèle

Propagation Avant (Forward Pass)

Propagation Avant = Le Modèle

Définition

La **propagation avant** (forward pass) décrit comment le réseau **calcule sa prédiction** à partir des entrées.

C'est le 1^{er} composant : le **Modèle** !

- 1 Les données x entrent par la couche d'entrée
- 2 Chaque couche transforme les données (poids + activation)
- 3 La sortie produit la prédiction $\hat{y}$

$$x \xrightarrow{\text{couche 1}} a^{(1)} \xrightarrow{\text{couche 2}} a^{(2)} \cdots \hat{y}$$

Analogie

Comme l'eau qui coule dans un réseau de tuyaux : l'entrée "propage" son signal jusqu'à la

Le Calcul Couche par Couche

Pour chaque couche l :

Étape 1 : Combinaison Linéaire

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

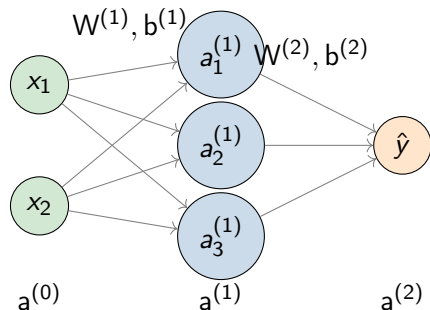
Étape 2 : Activation

$$a^{(l)} = \sigma(z^{(l)})$$

Notation :

- $a^{(0)} = x$ (entrée)
- $a^{(L)} = \hat{y}$ (sortie/prédiction)

Exemple : Réseau à 2 Couches



$$z^{(1)} = W^{(1)}x + b^{(1)} \rightarrow a^{(1)} = \text{ReLU}(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \rightarrow \hat{y} = \sigma(z^{(2)})$$

Que Contiennent les Matrices de Poids ?

Exemple : 2 entrées $\rightarrow$ 3 neurones cachés $\rightarrow$ 1 sortie

Couche 1 : Entrée $\rightarrow$ Cachée

$W^{(1)}$: matrice 3×2 $b^{(1)}$: vecteur taille 3

$$W^{(1)} = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{pmatrix} \quad b^{(1)} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

Couche 2 : Cachée $\rightarrow$ Sortie

$W^{(2)}$: matrice 1×3 $b^{(2)}$: scalaire

$$W^{(2)} = (w_1 \quad w_2 \quad w_3) \quad b^{(2)} = b$$

Règle Générale

$W^{(l)}$ a pour dimensions : **(neurones couche l) $\times$ (neurones couche $l - 1$)**

L'Algorithme Rétropropagation (Backpropagation)

Comment Entraîner un MLP ?

Le Principe (Rappel)

Comme pour les modèles linéaires : **descente de gradient** !

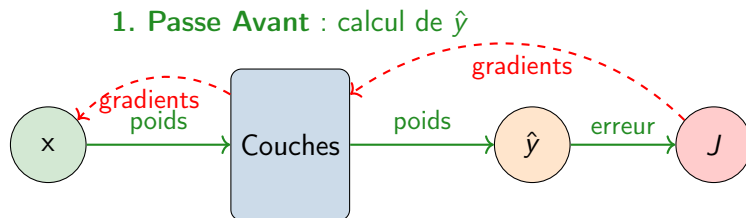
$$W \leftarrow W - \eta \cdot \nabla_W J(W)$$

Le Défi

Un MLP a **des milliers de poids** répartis sur plusieurs couches.
Comment calculer efficacement le gradient pour **chaque** poids ?

La Solution : Rétropropagation

Un algorithme efficace qui propage l'erreur de la **sortie vers l'entrée** pour calculer tous les gradients en une seule passe.



2. Passe Arrière : calcul des gradients

En Pratique

Les frameworks (PyTorch, TensorFlow) calculent la rétropropagation **automatiquement** !
Vous n'avez qu'à définir le modèle et appeler `loss.backward()`.

La Rétropropagation en Résumé

- ➊ **Passe Avant** : Les données traversent le réseau → Prédiction $\hat{y}$
- ➋ **Calcul de l'Erreur** : Mesurer $J(y, \hat{y})$ (MSE, Cross-Entropy...)
- ➌ **Passe Arrière** : Propager l'erreur pour calculer les gradients
- ➍ **Mise à Jour** : Ajuster les poids avec la descente de gradient

Algorithme inventé par Rumelhart, Hinton & Williams (1986) — a relancé les réseaux de neurones !

Pour Aller Plus Loin

Les détails mathématiques (règle de la chaîne, calcul des gradients couche par couche) seront vus dans un prochain cours.

	Perceptron	MLP
Structure	1 couche	3+ couches
Capacité	Séparation linéaire	Approximateur universel
Activation	Step function	ReLU, Sigmoid, etc.
Apprentissage	Règle du Perceptron	Rétropropagation
XOR	× Impossible	✓ Résolu

Des questions ?